

PokeBench: Reproducible Decision-Making Benchmarks in Pokemon Red

afk-sapien

October 2026 | Development technical report | Not peer reviewed

Abstract PokeBench evaluates language-model agents through reproducible checkpoint challenges in Pokemon Red. The gameplay track exposes structured player information and controller-backed shortcuts so that item selection, resource management, and navigation decisions matter more than repeated menu inputs. Private objective checks determine completion, while recorded controller traces support deterministic replay. We describe task curation, bounded context, usage accounting, paired starting conditions, and release evaluation controls. Audited development trials show differences between models, but they also show ceiling effects and sensitivity to the surrounding agent framework. Historical comparisons are therefore reported as development evidence, not as results of a newly frozen release. The report identifies the remaining requirements for a fair cross-provider comparison and publishes the analysis inputs separately from game assets and private model logs.

1. Introduction

A game-playing agent must choose useful actions, remember their consequences, and recover when its plan fails. A full Pokemon campaign combines navigation, team management, battles, and long-term objectives. It also creates an evaluation problem: an early navigation failure prevents observation of later battle skills. PokeBench uses checkpoint tasks to expose these skills independently, alongside longer tasks that retain their dependencies.

The intended construct is decision making within a specified agent framework. Structured observations and menu shortcuts reduce the burden of reading pixels and navigating repetitive menus. They also change the task. Results from this track cannot establish unaided visual gameplay ability or the intrinsic ability of a model independent of its tools. The benchmark must publish those tools, observation rules, and budgets with every result.

Historical development evidence covers 7 models. This report describes the current implementation and that evidence. The active catalog contains 28 tasks, of which 22 contribute to the historical development ranking. The remaining 6 are experimental candidates. A separate archive preserves 5 retired tasks. The public presentation can show one catalog without pretending that every task has the same validation status.

2. Methods

2.1. Environment and checkpoints

Each task starts from an emulator state with a declared objective. Fixture metadata record the game checksum, emulator and core versions, save checksum, setup procedure, and reference controller trace. Evaluation checks use private state and are kept outside the agent observation allowlist. A badge already present in the starting save does not count as new progress.

Curated battle tasks may deliberately alter the player's initial team, move power points, health, or inventory. These changes occur during offline fixture construction and are recorded in provenance. Opponent behavior remains game behavior. A reference replay proves that a particular controller trajectory can win from the checkpoint. It does not prove that a fixed strategy always wins, that the task is easy, or that the reference is optimal.

Only controller actions advance a scored run. Observations do not advance game time. Save loading, setup waits, and offline fixture edits are distinct from scored gameplay. A declared runtime and unchanged initial state are required for a deterministic replay claim.

2.2. Agent observations and actions

The structured gameplay track supplies the current screen and selected facts about the player, local map, visible interaction state, party, moves, inventory, and recent actions. Some facts are retrieved through tools instead of being repeated in every prompt. The observation contract must enumerate these fields. Private victory flags, future random outcomes, and reference policies are not agent inputs.

Actions include bounded movement, interaction, dialogue advancement, move selection, Pokemon switching, and item use. Shortcuts execute ordinary controller inputs and report the observed outcome. They do not choose the best item or Pokemon for the agent. Dialogue advancement collects continuation text and stops at choices that require a decision. This design tests whether the agent chooses a useful action while reducing the cost of issuing each intermediate button.

Navigation assistance also changes what is being tested. Local map facts make spatial planning more accessible than screenshot-only play. These observations must be identical across compared models. If a later implementation supplies a route plan or automatic battle strategy, it requires a separately identified protocol rather than silently replacing the present task.

2.3. Context and stopping rules

The runner maintains bounded interaction context and explicit working memory. Recent decisions, current facts, action feedback, and model-authored notes are retained according to a declared policy. A context rotation must preserve its recorded handoff rather than claim to preserve an unlimited conversation. Memory contents remain fallible agent statements. The evaluator must not repair a plan by inserting private task knowledge into that memory.

A token threshold applies to reported input plus output usage. Cached input is not added again on top of total input usage. A completed in-flight call can cross the threshold, so the threshold is not a guarantee of an exact maximum. Another decision is blocked after the threshold is reached. Additional limits on decisions, actions, frames, elapsed time, and prolonged lack of progress bound trials that would otherwise continue indefinitely.

A gameplay failure, budget stop, and infrastructure error are different outcomes. An invalid response or unavailable provider does not establish that the model lost the battle. Error handling preserves the original attempt and its known usage. A retry must have its own identifier and a stated eligibility rule. Unreported usage stays unknown rather than being replaced with zero.

2.4. Scoring and comparison

Task success is a private objective predicate, not a model's claim of success. Battle objectives require the relevant victory evidence. Capture objectives require the intended encounter to produce the added Pokemon. Resource and recovery tasks can forbid blackout, party replacement, or any fainting, depending on the declared task. Merely leaving a battle or reaching a map after a blackout cannot substitute for the specified objective.

For a release comparison, compute each model's success fraction within a task across the shared starting variants and registered attempts. Average those fractions across tasks so that a task with more variants does not receive more weight. Report the number of evaluated tasks and missing attempts beside the score. Experimental tasks remain visible but do not enter the primary ranking until the inclusion rule is satisfied.

The historical development ranking selects declared cohorts by task and retains matching coverage across models. Its retrospective curation and changing framework versions prevent treating that aggregate as a fresh controlled release experiment. Publication should preserve the historical table, then present a separate frozen release comparison without blending their results.

2.5. Randomness and calibration

Paired starts mean that every model receives the same registered checkpoints. They do not force identical outcomes after different actions. Action timing can change a deterministic emulator's later random sequence. Nearby timing variants also need not be independent random samples. A winning path found after a failure is evidence of feasibility, not a valid replacement for the failed trial.

Calibration compares a frozen reference policy with simple attack policies on held-out starts. Both reliability and separation matter. A task that a simple baseline reliably wins may be a useful basic control but adds little evidence of advanced tactical skill. Conversely, failure by every model can indicate a useful frontier challenge, an unreliable fixture, or an interface defect. Replay and reference checks are needed before interpreting the result.

2.6. Cost and provenance

Report token usage, decisions, controller actions, game frames, and active wall time alongside success. Subscription usage is not a direct cash invoice. Provider-rate estimates must carry their rate-card date, cache treatment, coverage, and units. Cross-provider token counts and effort settings are not necessarily equivalent measures of computation. A cost-to-success comparison must use matched tasks and complete accounting, with unknown cost shown explicitly.

The analysis consumes sanitized frozen summaries. It excludes game binaries, emulator saves, screenshots, prompts, responses, credentials, and local private paths. Input and artifact hashes support provenance checks but are not external attestation. This manuscript uses Paper Scaffold¹, which generates result tables and guards numerical claims against changes in their analysis inputs.

3. Development results

3.1. Audited decision-task pilot

The decision-task pilot contains 18 attempts across 6 tasks and 3 models. The exact provider model identifiers and reported usage appear in Table 1. Each task-model pair has one starting condition and one attempt. All included rows passed the replay, input-delivery, conversation-order, and state-update audits.

Astra completed 6 of 6 tasks. Terra completed 6 of 6, while Luna completed 2 of 6. These outcomes support testing the candidates further. They do not distinguish the stronger pair or estimate repeated-attempt reliability for an individual task. Aggregate pilot usage was 4,985,443 reported tokens.

Model identifier	Passed	Reported tokens	Replay and delivery
gpt-5.6-luna	2/6	1,580,120	All verified
gpt-5.6-terra	6/6	1,941,325	All verified
gpt-6-astra	6/6	1,463,998	All verified

Table 1: Audited development pilot. The pass denominator is distinct tasks, not repeated trials of one task. Token totals include reported input and output usage. This is not the frozen release evaluation.

3.2. Reference-policy evidence

The initial decision-task calibration sweep is shown in Table 2. Reference wins varied across the shared starts, despite each released fixture having a winning reference trajectory. Healthy capture was sometimes successful with immediate ball throwing. These observations caution against labeling every candidate a difficult reasoning test. The route reference used the simulation’s privileged offline policy and is not a model result.

Experimental task	Reference	Simple baseline
Recover from a bad matchup	3/5	0/5
Capture a healthy Snorlax	5/5	3/5
Rescue a damaged team before Lance	2/5	0/5
Win Lorelei and Bruno with limited PP	4/5	0/5
Defeat the Champion with a specialist team	5/5	0/5
Reach Viridian Forest with a poisoned partner	5/5	0/5

Table 2: Development calibration of experimental tasks on shared timing variants. Reference strategies and simple baselines were fixed for this sweep. These are development starts, not held-out reliability estimates. Recovery-detour results compare a privileged simulation reference with its travel path without the recovery stop.

The separate tactical Lorelei candidate had stronger held-out evidence. Its frozen reference won 31 of 32 starts. The direct attacker won 0, and the attacker using type information won 0. The remaining reference loss and a later successful diagnostic trajectory are both retained. The diagnostic does not change the calibration score or establish perfect play.

4. Frozen release evaluation

Controlled release results are pending. The registered protocol pokebench-v1-beta freezes 16 model identifiers and 28 tasks, with 3 fresh timing variants per task. This creates 1,344 planned attempts. No development attempt is imported into these prospective results. The protocol records source, checkpoint, game, prompt, response-schema, and fixture-registration hashes.

The approved aggregate budget is 50,000,000 reported tokens. The sum of all per-attempt thresholds is 1,260,000,000 tokens, so the registered matrix is a staged plan rather than a promise to finish every attempt under the approved cap. A task-first schedule evaluates paired variants and rotates model order within each variant. Only eligible tasks with identical completed, replay-verified and accounted variant coverage across all registered models enter the aggregate. Incomplete coverage stays visible. A partial aggregate must state which tasks it covers and cannot represent the entire catalog.

Both providers use the gameplay track, the same prompt and response-schema hashes, and bounded text observations. The context policy retains at most 8 turns per segment and rotates at the declared 12,000 context threshold. It requests 0 paid summaries. Deterministic checkpoint packets carry current state and retained memory between segments. The protocol records transport and memory-policy identifiers so that these choices can be reproduced.

The provider model identifiers appear in Table 4. Requested effort is medium. Haiku exposes no adjustable effort, which is recorded as unavailable rather than medium. Claude Code has a 4,096 output-token limit per call, while the Codex transport records no adapter output cap. These are declared provider differences, not evidence of equal computational effort. Claude access was listed by the official client, with representative smoke tests recorded separately from task completion.

No gameplay retries or paid fallback are permitted. Infrastructure errors and incomplete accounting stop the sweep, while every raw attempt is retained. The controller reserves 100,000 tokens for an in-flight response, but one response can still cross a decision-boundary threshold. Per-task Wilson intervals are descriptive because timing variants are paired conditions rather than independent random samples. Only 22 tasks are eligible for the primary score. Experimental tasks remain excluded. Release outcomes must be reported with these limits before making claims about relative ability.

5. Limitations

PokeBench evaluates a model with an agent framework, not an isolated model. Changing summaries, memory retention, menu automation, or map observations can change performance. An ablation can measure the

effect of a particular change only when the other conditions are held fixed. The earlier development sequence changed several components together and does not provide that causal evidence.

The tasks sample one game and a curated set of situations. Models may have seen Pokemon strategies during training. Success can reflect recalled knowledge, online planning, or both. Task selection after observing failures can overfit the evaluated model set. Development and final evaluation starts should therefore remain distinct, and later task changes should create a new release.

Stochastic outcomes limit the meaning of a single win or loss. Deterministic replay verifies execution but does not remove this evaluation uncertainty. Ceiling effects remain in the pilot, and several reference policies are not reliably successful across candidate starts. The report does not claim that the current catalog separates every model tier or proves human-level gameplay.

6. Availability and next validation

A release should provide source code, a machine-readable protocol, sanitized attempt summaries, analysis commands, and the technical report. The website should show one catalog with task status, model coverage, results, and replay links where distributable. Users must supply a compatible game ROM. Game assets and private account records are not part of the source distribution.

The registered validation is a staged cross-provider run with shared fresh starts. Its results can replace the pending release section through the same audited export and manuscript build. The earlier manuscript sources and all development inputs remain archived so readers can trace how the protocol changed.

Bibliography

- (1) Garrett, P. T. *paper-scaffold: provenance-checked Typst manuscripts*. <https://doi.org/10.5281/zenodo.22905759>.

7. Supporting Information

7.1. Current development catalog

The task inventory appears in Table 3.

Task	Category	Development status
Get a starter	Navigation	Scored
Heal the party	Recovery	Scored
Collect and deliver Oak's Parcel	Navigation	Scored
Recover the party with items	Recovery	Scored
Defeat Agatha	Battles	Scored
Defeat Lance	Battles	Scored
Become Champion	Campaign	Scored
Return Oak's Parcel from Viridian	Navigation	Scored
Defeat Lorelei	Battles	Scored
Defeat Bruno	Battles	Scored
Win the final Champion battle	Battles	Scored
Buy ten Poke Balls	Shopping	Scored
Cross Viridian Forest to Pewter	Navigation	Scored
Reach Cerulean through Mt. Moon	Navigation	Scored
Win a fossil in Mt. Moon	Collection	Scored
Defeat Misty	Battles	Scored
Defeat Lt. Surge	Battles	Scored
Defeat Koga	Battles	Scored
Defeat Blaine	Battles	Scored
Defeat Giovanni	Battles	Scored
Become Champion with a difficult team	Campaign	Scored
Lorelei with a tactical control team	Battles	Scored
Recover from a bad matchup	Battles	Experimental
Capture a healthy Snorlax	Collection	Experimental
Rescue a damaged team before Lance	Recovery	Experimental
Win Lorelei and Bruno with limited PP	Battles	Experimental
Defeat the Champion with a specialist team	Battles	Experimental
Reach Viridian Forest with a poisoned partner	Navigation	Experimental

Table 3: Task inventory at the frozen development snapshot. Scored means included in the historical development ranking. It does not imply that fresh release trials are complete. Experimental candidates remain outside that ranking.

7.2. Reproduction and disclosure

The report statistics and tables are generated from sanitized inputs under analysis/inputs. Run just assets, just paper, and just verify from the manuscript directory. The export script records hashes of its source summaries and selects an explicit set of public fields. Re-exporting data changes those hashes and requires a new build.

The archived pre-release manuscript contains earlier observations of visual perception, navigation, memory, and budget failures. Those studies used different framework versions and are not pooled into the current pilot. The upstream Paper Scaffold license and pinned toolchain revision are preserved in the paper directory. Reported results do not imply endorsement by the game publisher or model providers.

7.3. Registered release models

Table 4 lists the prospective matrix. Registration is distinct from completed evaluation and from a successful provider-access smoke test.

Provider	Registered model identifier	Effective effort
codex	gpt-6-astra	medium
codex	gpt-5.6-sol	medium
codex	gpt-6-sol	medium
codex	gpt-5.5	medium
codex	gpt-5.6-terra	medium
codex	gpt-5.6-luna	medium
codex	gpt-6-luna	medium
claude	claude-opus-5-5	medium
claude	claude-sonnet-5-5	medium
claude	claude-haiku-4-5-20251001	Not adjustable
claude	claude-sonnet-5	medium
claude	claude-opus-5	medium
claude	claude-opus-4-8	medium
claude	claude-opus-4-7	medium
claude	claude-opus-4-6	medium
claude	claude-sonnet-4-6	medium

Table 4: Registered identifiers and effort settings in the prospective beta protocol. These are planned participants, not results.

The complete frozen protocol is supplied as `analysis/inputs/pokebench-v1-protocol.json`. Its canonical checksum is `99c4e3a7dfbc0598d2be7c2f75201a08efeb7c63ba7b617da30bc4c7ad2a6d05`. It records exact task thresholds, starting-state hashes, runtime identifiers, transport settings, and scoring rules. The source protocol was checked against this checksum before inclusion in the public paper inputs.